simple; you're using fine-grained locking because you want the maximum possible amount of concurrency. If you leave the mutex locked across the call to `notify_one()` (as in listing 6.2), then if the notified thread wakes up before the mutex has been unlocked, it will have to wait for the mutex. On the other hand, if you unlock the mutex *before* you call `notify_one()`, then the mutex is available for the waiting thread to acquire when it wakes up (assuming no other thread locks it first). This is a minor improvement, but it might be important in some cases.

`wait_and_pop()` is more complicated, because you have to decide where to wait, what the predicate is, and which mutex needs to be locked. The condition you're waiting for is "queue not empty," which is represented by `head!=tail`. Written like that, it would require both `head_mutex` and `tail_mutex` to be locked, but you've already decided in listing 6.6 that you only need to lock `tail_mutex` for the read of `tail` and not for the comparison itself, so you can apply the same logic here. If you make the predicate `head!=get_tail()`, you only need to hold the `head_mutex`, so you can use your lock on that for the call to `data_cond.wait()`. Once you've added the wait logic, the implementation is the same as `try_pop()`.

The second overload of `try_pop()` and the corresponding `wait_and_pop()` overload require careful thought. If you just replace the return of the `std::shared_ptr<>` retrieved from `old_head` with a copy assignment to the `value` parameter, there's a potential exception-safety issue. At this point, the data item has been removed from the queue and the mutex unlocked; all that remains is to return the data to the caller. However, if the copy assignment throws an exception (as it very well might), the data item is lost because it can't be returned to the queue in the same place.

If the actual type `T` used for the template argument has a no-throw move-assignment operator or a no-throw swap operation, you could use that, but you'd really like a general solution that could be used for any type `T`. In this case, you have to move the potential throwing inside the locked region, before the node is removed from the list. This means you need an extra